

Functions and Recursion in C

> **Definition to Remember**

1. Concept Overview & Fundamentals

Functions and Recursion in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization

```text

### Core Architecture Breakdown

| Component / Concept | Technical Specification | Operational Characteristics | Performance / Complexity |

| :--- | :

---

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
```c
#include <stdio.h>

// Core standard implementation pattern
int main() {

// Core logic execution
printf("Core Logic Execution\n");

return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

- **E**

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: add function

Problem Description & Objective:

****C Code Implementation:****

```
#include <stdio.h>
```

```
// Function Definition
```

```
int main() {
```

```
int result;
```

```
result = add(10, 20, 30, 40);
```

```
printf("Sum = %d\n", result);
```

```
return 0;
```

```
---
```

Problem 2: fibonacci recursion

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
// Tail-recursive helper function to calculate Fibonacci in O(n) time
```

```
// Recursive Function Wrapper
```

```
int main() {
```

```
    printf("Enter number of terms: ");
```

```
    if (n < 0) {
```

```
        printf("Fibonacci Series:\n");
```

```
        for(i = 0; i < n; i++) {
```

```
            return 0;
```

```
    ---
```

Problem 3: get average function

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
// Function Definition
```

```
return (a + b + c + d + e) / 5.0;
```

```
int main() {
```

```
float average;
```

```
average = get_average(10, 20, 30, 40, 50);
```

```
printf("Average = %.2f\n", average);
```

```
return 0;
```

```
---
```

Problem 4: greet function

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
// Function Definition
```

```
int main() {
```

```
// Function Call
```

```
return 0;
```

```
---
```

Problem 5: increment function

****Problem Description & Objective:****

****C Code Implementation:****

```
#include <stdio.h>
```

```
// Function Definition
```

```
printf("Value inside function = %d\n", num);
```

```
int main() {
```

```
int number = 10;
```

```
printf("Original value before function call = %d\n", number);
```

```
increment(number);
```

```
printf("Original value after function call = %d\n", number);
```

```
return 0;
```

```
}
```

```
---
```

Problem 6: max function

****Problem Description & Objective:****

Implemen

****C Code Implementation:****

```
```c
```

```
#include <stdio.h>
```

```
// Function Definition
```

```
float m
```

```
if(a > b) {
```

```
return
```

```
int main() {
```

```
/*
```

```
float num1, num2, result;
```

```
printf("Enter first number: ");
```

```
scanf(
```

```
printf("Enter second number: ");
```

```
scanf(
```

```
result = max(num1, num2);
```

```
printf("Larger number = %.2f\n", result);
```

```
return 0;
```

```
}
```

```

```

**Problem 7: palindrome recursion****\*\*Problem Description & Objective:\*\***

Implemen

**\*\*C Code Implementation:\*\***

```c

#include <stdio.h>

// Recursive Function

int rev

if(num == 0) {

return

return reverseNumber(num / 10, rev * 10 + num % 10);

}

int main() {

/*

int num, reversed;

printf("Enter a number: ");

scanf(")

reversed = reverseNumber(num, 0);

if(num == reversed) {

printf(")

return 0;

}

Problem 8: print date function****Problem Description & Objective:****

Implemen

****C Code Implementation:****

```
#include <stdio.h>
```

```
// Function Definition
```

```
int main() {
```

```
// Function Call
```

```
return 0;
```

```
---
```

Problem 9: square function

****Problem Description & Objective:****

****C Code Implementation:****

Define a function square that takes an int

```
#include <stdio.h>
```

```
// Function Definition
```

```
int main() {
```

```
int number, result;
```

```
printf("Enter a number: ");
```

```
scanf("%d", &number);
```

```
result = square(number);
```

```
printf("Square = %d\n", result);
```

```
return 0;
```

```
}
```

```
---
```

6. Exam Scoring Points & Memory Revision

> **Must-Write Points**

> 1. Full Marks

> **Quick Recall**

> `Pro`